

Lab 05

AUTHOR

Leticia Rocha Soares Martins [221324]

Pacotes

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1    ✓ readr      2.2.0
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2    4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr      1.2.2

— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
! Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(leaflet)
library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

discard

The following object is masked from 'package:readr':

col_factor

Especificação das colunas do dataset flights

Vamos especificar as colunas do dataset para facilitar a leitura dos dados.

```
TIPOS_COLUNAS_VOOS <- cols(
  YEAR           = col_integer(),
  MONTH          = col_integer(),
  DAY            = col_integer(),
  DAY_OF_WEEK    = col_integer(),
  AIRLINE        = col_character(),
  FLIGHT_NUMBER  = col_integer(),
  TAIL_NUMBER    = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_integer(),
  DEPARTURE_TIME = col_integer(),
```

```

DEPARTURE_DELAY      = col_double(),
TAXI_OUT              = col_double(),
WHEELS_OFF           = col_integer(),
SCHEDULED_TIME       = col_double(),
ELAPSED_TIME         = col_double(),
AIR_TIME             = col_double(),
DISTANCE             = col_double(),
WHEELS_ON            = col_integer(),
TAXI_IN              = col_double(),
SCHEDULED_ARRIVAL    = col_integer(),
ARRIVAL_TIME         = col_integer(),
ARRIVAL_DELAY        = col_double(),
DIVERTED              = col_integer(),
CANCELLED            = col_integer(),
CANCELLATION_REASON  = col_character(),
AIR_SYSTEM_DELAY     = col_double(),
SECURITY_DELAY       = col_double(),
AIRLINE_DELAY        = col_double(),
LATE_AIRCRAFT_DELAY  = col_double(),
WEATHER_DELAY        = col_double()
)

```

Funções auxiliares

```

#' Converte os horários dos voos
converte_hhmm_para_duracao <- function(horario_hhmm) {
  horario_hhmm <- if_else(horario_hhmm == 2400L, 0L, horario_hhmm)
  horas      <- horario_hhmm %/% 100L
  minutos    <- horario_hhmm %% 100L
  dhours(horas) + dminutes(minutos)
}

#' Le o arquivo dos voos por parte
le_voos_da_aeronave <- function(arquivo, tail_number, tamanho_bloco = 100000L) {

  filtra_bloco <- function(bloco_de_voos, posicao_inicial) {
    filter(bloco_de_voos, TAIL_NUMBER == tail_number)
  }

  read_csv_chunked(
    file      = file.path("flights.csv"),
    callback  = DataFrameCallback$new(filtra_bloco),
    chunk_size = tamanho_bloco,
    col_types = TIPOS_COLUNAS_VOOS,
    progress  = FALSE
  )
}

#' Lê o arquivo de aeroportos
le_aeroportos <- function(arquivo_aeroportos) {
  read_csv(
    arquivo_aeroportos,
    col_types = cols(

```

```

    IATA_CODE = col_character(),
    AIRPORT   = col_character(),
    CITY      = col_character(),
    STATE     = col_character(),
    COUNTRY   = col_character(),
    LATITUDE  = col_double(),
    LONGITUDE = col_double()
  )
) %>%
  select(IATA_CODE, NOME_AEROPORTO = AIRPORT, CIDADE = CITY,
         LATITUDE, LONGITUDE)
}

```

Função principal

```

#' Analisa o trajeto anual de uma aeronave
analisa_aeronave <- function(tail_number,
                             arquivo,
                             arquivo_aeroportos = file.path(dirname(arquivo),
                                                                "airports.csv"),
                             tamanho_bloco = 100000L) {

  ## Validação dos argumentos
  stopifnot(is.character(tail_number), length(tail_number) == 1L)
  if (!file.exists(arquivo))
    stop("Arquivo de voos não encontrado: ", arquivo)
  if (!file.exists(arquivo_aeroportos))
    stop("Arquivo de aeroportos não encontrado: ", arquivo_aeroportos)

  ## Leitura por partes
  voos_da_aeronave <- le_voos_da_aeronave(arquivo, tail_number, tamanho_bloco)

  if (nrow(voos_da_aeronave) == 0L)
    stop("Nenhum voo encontrado para a aeronave ", tail_number)

  aeroportos <- le_aeroportos(arquivo_aeroportos)

  ## Construção da tabela tidy
  ## Mantém TODAS as colunas originais e acrescenta as derivadas.
  voos_ordenados <- voos_da_aeronave %>%
    filter(CANCELLED == 0L, DIVERTED == 0L, !is.na(DEPARTURE_TIME)) %>%
    filter(str_detect(ORIGIN_AIRPORT, "[A-Z]{3}$"),
           str_detect(DESTINATION_AIRPORT, "[A-Z]{3}$")) %>%
    mutate(
      data_do_voo          = make_date(YEAR, MONTH, DAY),
      instante_partida     = as_datetime(data_do_voo) +
        converte_hhmm_para_duracao(SCHEDULED_DEPARTURE),
      velocidade_media_mph = if_else(AIR_TIME > 0,
                                      DISTANCE / (AIR_TIME / 60), NA_real_)
    ) %>%
    arrange(instante_partida) %>%
    mutate(ordem_no_trajeto = row_number())
}

```

```

trajetos_com_coordenadas <- voos_ordenados %>%
  inner_join(aeroportos %>%
    rename(ORIGIN_AIRPORT = IATA_CODE,
           nome_origem     = NOME_AEROPORTO,
           cidade_origem   = CIDADE,
           latitude_origem  = LATITUDE,
           longitude_origem = LONGITUDE),
    by = "ORIGIN_AIRPORT") %>%
  inner_join(aeroportos %>%
    rename(DESTINATION_AIRPORT = IATA_CODE,
           nome_destino         = NOME_AEROPORTO,
           cidade_destino       = CIDADE,
           latitude_destino     = LATITUDE,
           longitude_destino    = LONGITUDE),
    by = "DESTINATION_AIRPORT") %>%
  arrange(instante_partida) %>%
  mutate(ordem_no_trajeto = row_number())

## Aeroportos visitados
aeroportos_visitados <- trajetos_com_coordenadas %>%
  select(IATA_CODE = ORIGIN_AIRPORT, nome = nome_origem,
         cidade = cidade_origem,
         latitude = latitude_origem, longitude = longitude_origem) %>%
  bind_rows(
    trajetos_com_coordenadas %>%
      select(IATA_CODE = DESTINATION_AIRPORT, nome = nome_destino,
             cidade = cidade_destino,
             latitude = latitude_destino, longitude = longitude_destino)
  ) %>%
  count(IATA_CODE, nome, cidade, latitude, longitude,
        name = "numero_de_pousos_e_decolagens")

## Escalas visuais
## Espessura da linha -> velocidade média do trecho (requisito d)
## Cor da linha       -> atraso na chegada (estatística adicional)
espessura_da_linha <- rescale(trajetos_com_coordenadas$velocidade_media_mph,
                             to = c(1, 8))
espessura_da_linha[is.na(espessura_da_linha)] <- 1

paleta_de_atraso <- colorNumeric(
  palette = "RdYlGn",
  domain = trajetos_com_coordenadas$ARRIVAL_DELAY,
  reverse = TRUE
)

## Mapa
mapa_do_trajeto <- leaflet(trajetos_com_coordenadas) %>%
  addProviderTiles(providers$CartoDB.Positron)

for (indice_do_trecho in seq_len(nrow(trajetos_com_coordenadas))) {

  trecho <- trajetos_com_coordenadas[indice_do_trecho, ]

  texto_do_popup <- str_c(

```

```

    "<b>Trecho ", trecho$order_no_trajeto, "</b><br/>",
    trecho$ORIGIN_AIRPORT, " &rarr; ", trecho$DESTINATION_AIRPORT, "<br/>",
    "Data: ", format(trecho$instante_partida, "%d/%m/%Y %H:%M"), "<br/>",
    "Cia: ", trecho$AIRLINE, " / voo ", trecho$FLIGHT_NUMBER, "<br/>",
    "Distância: ", round(trecho$DISTANCE), " milhas<br/>",
    "Velocidade média: ", round(trecho$velocidade_media_mph), " mph<br/>",
    "Atraso na chegada: ", trecho$ARRIVAL_DELAY, " min"
  )

  mapa_do_trajeto <- mapa_do_trajeto %>%
    addPolylines(
      lng   = c(trecho$longitude_origem, trecho$longitude_destino),
      lat   = c(trecho$latitude_origem,  trecho$latitude_destino),
      weight = espessura_da_linha[indice_do_trecho],
      color  = paleta_de_atraso(trecho$ARRIVAL_DELAY),
      opacity = 0.6,
      popup  = texto_do_popup
    )
}

## Marcadores dos aeroportos: raio proporcional ao número de operações
mapa_do_trajeto <- mapa_do_trajeto %>%
  addCircleMarkers(
    data   = aeroportos_visitados,
    lng    = ~longitude,
    lat    = ~latitude,
    radius = ~rescale(numero_de_pousos_e_decolagens, to = c(3, 10)),
    color  = "#1f2d3d",
    fillOpacity = 0.8,
    stroke = FALSE,
    label  = ~str_c(IATA_CODE, " - ", cidade,
                    " (", numero_de_pousos_e_decolagens, " operações)")
  ) %>%
  addLegend(
    position = "bottomright",
    pal      = paleta_de_atraso,
    values   = trajetos_com_coordenadas$ARRIVAL_DELAY,
    title    = str_c("Aeronave ", tail_number, "<br/>Atraso (min)"),
    opacity  = 0.8
  )

## Lista
list(
  tabela = trajetos_com_coordenadas,
  grafico = mapa_do_trajeto
)
}

```

Resultado

```

resultado <- analisa_aeronave(
  tail_number = "N431WN",

```

```
arquivo = file.path("flights.csv")
)
```

Warning in validateCoords(lng, lat, funcName): Data contains 1 rows with either missing or invalid lat/lon values and will be ignored

```
resultado$tabela
```

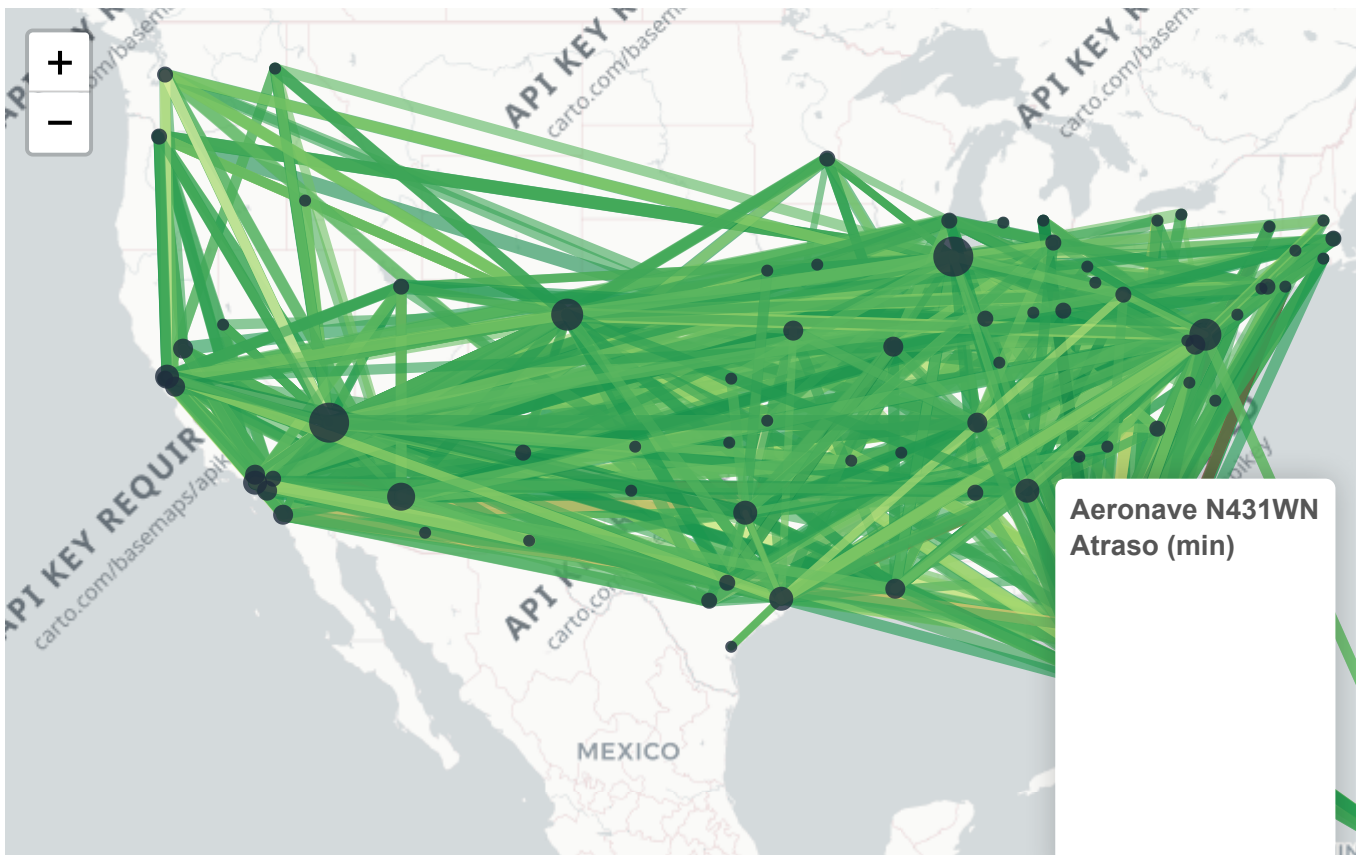
A tibble: 1,768 × 43

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER
	<int>	<int>	<int>	<int>	<chr>	<int>	<chr>
1	2015	1	1	4	WN	1762	N431WN
2	2015	1	1	4	WN	3564	N431WN
3	2015	1	1	4	WN	3564	N431WN
4	2015	1	1	4	WN	3564	N431WN
5	2015	1	2	5	WN	1793	N431WN
6	2015	1	2	5	WN	1793	N431WN
7	2015	1	2	5	WN	1793	N431WN
8	2015	1	2	5	WN	1793	N431WN
9	2015	1	2	5	WN	1793	N431WN
10	2015	1	2	5	WN	3144	N431WN

i 1,758 more rows

i 36 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
 # SCHEDULED_DEPARTURE <int>, DEPARTURE_TIME <int>, DEPARTURE_DELAY <dbl>,
 # TAXI_OUT <dbl>, WHEELS_OFF <int>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
 # AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <int>, TAXI_IN <dbl>,
 # SCHEDULED_ARRIVAL <int>, ARRIVAL_TIME <int>, ARRIVAL_DELAY <dbl>,
 # DIVERTED <int>, CANCELLED <int>, CANCELLATION_REASON <chr>, ...

```
resultado$grafico
```





```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 22:41:07 -03"
```